Université Abdelmalek Essaâdi
Faculté des sciences et techniques de Tanger
جامعة عبد المالك السعدي
كلية العلوم والتقنيات

RAPPORT DE PROJET DE FIN DE MODULE

Filière : Licence IDAI

Application Web Django : PreSkool

PROJET DE FIN DE MODULE : SYSTEME DE GESTION SCOLAIRE

Encadré par :

Prof. Sara AHSAIN

Réalisé par :

ZIOUANI Doha
CHENTOUF Ayoub

Dépôt GitHub : `https://github.com/DohaZiouani/school-django`

Année scolaire : 2025 – 2026

Résumé

Ce rapport présente la conception et la réalisation d'une application web de gestion scolaire nommée PreSkool, développée avec Django/Python et une base de données SQLite. L'objectif est de fournir une plateforme centralisée permettant la gestion des étudiants et de leurs parents, le suivi des enseignants et des départements, la planification des matières et des examens, ainsi que la consultation du calendrier des jours fériés. L'application organise ces fonctionnalités autour de trois profils d'accès : un espace administrateur pour la supervision complète, un espace enseignant pour la gestion pédagogique, et un espace étudiant pour la consultation personnalisée.

Au-delà de l'implémentation technique, le projet adopte une démarche structurée intégrant la modélisation de la base de données, la séparation des responsabilités selon l'architecture MVT de Django, et l'intégration de mécanismes clés tels que la gestion des rôles via un décorateur personnalisé (role required), un modèle utilisateur étendu (CustomUser héritant de AbstractUser), ainsi que la relation OneToOneField entre les modèles Student et Parent. L'administration Django permet la gestion complète des entités avec recherche, filtres et affichage personnalisé.

Table des matières

Glossaire **5**

1 Introduction **8**
1.0.1 Contexte général du projet : Application Web de Gestion Scolaire PreSkool . . . 8
1.0.2 Objectif du Projet . . . 8
1.0.3 Problématique . . . 9
1.0.4 Les Outils utilisés . . . 10

2 Conception et Développement **11**
2.1 Conception et Développement de l'Application Web de Gestion Scolaire PreSkool . . . 11
2.1.1 Objectif et principes de conception . . . 11
2.2 Diagrammes UML . . . 11
2.2.1 Diagramme de classes . . . 11
2.2.2 Structure du projet (résumé) . . . 13
2.2.3 Fichiers importants . . . 14

3 Cahier des charges **15**
3.1 Contexte et besoin . . . 15
3.1.1 Objectifs . . . 15
3.1.2 Acteurs et rôles . . . 15
Étudiant (Student) . . . 15
Enseignant (Teacher) . . . 16
Administrateur (Admin) . . . 16
3.1.3 Fonctionnalités principales . . . 17
Gestion administrative et pédagogique (Catalogue) . . . 17
Système d'évaluation et gestion de l'Emploi du temps . . . 17
Tableaux de bord interactifs (Dashboards) et statistiques . . . 17
Sécurité et Contrôle d'Accès basé sur les Rôles (RBAC) . . . 17
3.1.4 Fonctionnalités implémentées . . . 17
3.1.5 Choix technologiques . . . 18
Le framework backend : Django (Python) . . . 18
Base de données relationnelle (SGBDR) . . . 18

Frontend et utilisation de bibliothèques tierces 19
Communication Backend-Frontend : Sérialisation JSON 19

4 Réalisation **20**
4.1 Implémentation 20
4.1.1 Architecture du projet (MVT) 20
4.1.2 Fonctionnalités principales 20
4.1.3 Règles de gestion importantes 21
Gestion des inscriptions et évaluations 21
Cohérence de l'emploi du temps (Timetable) 22
Permissions (RBAC - Role-Based Access Control) 24
4.2 Sécurité et contrôle d'accès 26
4.2.1 Décorateurs d'authentification 26
4.2.2 Rôles et permissions (CustomUser) 26
4.2.3 Protection des routes d'administration 27
4.2.4 Validation et traitement des formulaires 28
4.2.5 Gestion des sessions 28
4.3 Tests et scénarios 28
4.3.1 Scénarios de test 28

5 Difficultés rencontrées **30**
5.1 Erreurs 30
Affichage statique et désorganisé du calendrier dans le tableau de bord étudiant 30
Affichage désorganisé du menu latéral et des fichiers statiques 31
5.1.1 1. Conflits d'architecture JavaScript et héritage des Templates (Le bug du "#") 31
5.1.2 2. Sécurisation stricte des routes et gestion des rôles (RBAC) 32
5.1.3 3. Intégration dynamique de l'emploi du temps (FullCalendar.js) . . 32
5.1.4 4. Cohérence relationnelle dans le module d'évaluation 33

6 Conclusion générale **34**
6.1 Conclusion 34

Liste des figures

2.1 Diagramme de classes du système de gestion scolaire (School Management System), présentant les entités principales (*CustomUser, Teacher, Student, Subject, Exam, etc.*), leurs attributs, et leurs relations (gestion des inscriptions, suivi des examens et emploi du temps). 12

3.1 Les rôles de l'application PreSkool 16

4.1 Aperçu du tableau de bord dynamique présentant les statistiques globales . . 21
4.2 Interface du portail de notation permettant aux enseignants de saisir rapidement les notes et les commentaires de toute une classe pour un examen spécifique 21
4.3 Interface du tableau de bord des examens présentant le nouveau bouton d'accès au portail de notation. 22
4.4 Interface du planning des cours générée via FullCalendar 22
4.5 Planifiez de nouveaux cours en sélectionnant la matière, le jour, les heures de début et de fin, ainsi que la salle, puis cliquez sur 'Save to Timetable' pour mettre à jour l'emploi du temps visuel 23
4.6 Le contrôle d'accès : view admin 24
4.7 Le contrôle d'accès : view teacher 25
4.8 Le contrôle d'accès : view student 26
4.9 Interception d'une tentative d'accès non autorisée par le système de sécurité 27
4.10 Interface d'authentification illustrant la gestion sécurisée des erreurs avec une alerte 'Invalid credentials' pour bloquer les tentatives de connexion incorrectes. 27
4.11 Validation du scénario d'administration : Formulaire d'ajout d'un étudiant . 28
4.12 Formulaire d'ajout du note d'un étudiant 29

5.1 Calendrier statique et mal affiché avant correction 30
5.2 Interface sans style avant correction des fichiers statiques 31

Glossaire

Django
Framework web Python de haut niveau suivant l'architecture MVT (Model-View-Template), utilisé pour développer l'application PreSkool. Il fournit un ORM, une interface d'administration automatique et un système d'authentification intégré.

MVT (Model-View-Template)
Architecture logicielle utilisée par Django, équivalente au patron MVC. Le Model gère les données, la View contient la logique métier, et le Template assure la présentation HTML.

ORM (Object-Relational Mapper)
Outil Django permettant de manipuler la base de données en écrivant du code Python plutôt que du SQL. Chaque classe Python correspond à une table de la base de données.

Model
Classe Python Django définissant la structure d'une entité de la base de données. Dans PreSkool, les modèles principaux sont : Student, Parent, Teacher, Department, Subject, Exam, Holiday et CustomUser.

View
Fonction Python Django recevant une requête HTTP et retournant une réponse. Les vues de PreSkool sont protégées par le décorateur `@role_required` selon le rôle de l'utilisateur.

Template
Fichier HTML Django utilisant la syntaxe `{% %}` et `{{ }}` pour afficher dynamiquement les données. PreSkool utilise le template Bootstrap PreSkool avec héritage via `base.html`.

Migration
Fichier Python généré automatiquement par Django (`makemigrations`) décrivant les modifications à apporter à la base de données. La commande `migrate` applique ces modifications.

CustomUser : Modèle utilisateur personnalisé héritant de `AbstractUser` de Django, enrichi des champs `is_admin`, `is_teacher` et `is_student` pour gérer les trois rôles de l'application.

AbstractUser
Classe Django permettant d'étendre le modèle utilisateur par défaut. Son utilisation nécessite de définir `AUTH_USER_MODEL` dans `settings.py` avant la première migration.

Décorateur (Decorator)
Fonction Python modifiant le comportement d'une autre fonction. Dans PreSkool, `@role_required` vérifie le rôle de l'utilisateur avant d'autoriser l'accès à une vue.

OneToOneField
Type de relation Django établissant un lien unique entre deux modèles. Dans PreSkool, chaque Student possède exactement un Parent et vice versa. La suppression du parent entraîne la suppression de l'étudiant (CASCADE).

ForeignKey
Type de relation Django établissant un lien entre plusieurs enregistrements. Par exemple, plusieurs Teacher peuvent appartenir au même Department, et plusieurs Subject peuvent être associées au même Teacher.

SQLite
Système de gestion de base de données légère utilisé par défaut dans Django pour le développement. Ne nécessite pas de serveur séparé, les données sont stockées dans un fichier `db.sqlite3`.

Bootstrap
Framework CSS/JavaScript utilisé pour concevoir l'interface graphique de PreSkool. Le template PreSkool fournit un design professionnel et responsive avec menu latéral, tableaux et graphiques.

Fichiers statiques (Static Files)
Fichiers CSS, JavaScript et images servis directement par Django sans traitement. Configurés via `STATICFILES_DIRS` dans `settings.py` et référencés dans les templates avec `{% static '...' %}`.

Environnement virtuel (Virtual Environment)
Espace Python isolé créé avec `python -m venv monenv` permettant d'installer les dépendances du projet sans affecter le système global.

requirements.txt
Fichier listant toutes les dépendances Python du projet avec leurs versions exactes. Généré avec `pip freeze > requirements.txt` et utilisé pour reproduire l'environnement avec `pip install -r requirements.txt`.

CRUD
Acronyme désignant les quatre opérations de base sur les données : Create (créer), Read (lire), Update (modifier), Delete (supprimer). PreSkool implémente le CRUD complet pour les étudiants, enseignants, départements, matières et examens.

Superuser
Compte administrateur Django créé avec `python manage.py createsuperuser`, ayant accès à l'interface d'administration (`/admin/`) et bypassant automatiquement toutes les vérifications de rôle.

GitHub
Plateforme de gestion de code source basée sur Git. Utilisée dans ce projet pour la collaboration en binôme, le versionnage du code et la soumission du projet au professeur via un dépôt public.

ImageField
Champ Django permettant l'upload et le stockage d'images. Utilisé dans PreSkool pour les photos des étudiants (`upload_to='students/'`) et des enseignants (`upload_to='teachers/'`). Nécessite la bibliothèque Pillow.

Pillow
Bibliothèque Python de traitement d'images, requise par Django pour utiliser les champs `ImageField`. Installée via `pip install Pillow`.

Chapitre 1

Introduction

1.0.1 Contexte général du projet : Application Web de Gestion Scolaire PreSkool

Le projet *Application Web de Gestion Scolaire PreSkool* s'inscrit dans le cadre du développement d'une plateforme numérique centralisée destinée à moderniser la gestion administrative et pédagogique d'un établissement scolaire fictif. La gestion d'une école moderne implique la manipulation d'un ensemble complexe de données : étudiants, parents, enseignants, départements, matières, examens et calendriers. Actuellement, de nombreux établissements font face à des difficultés pour centraliser ces informations, ce qui peut entraîner des erreurs administratives, une perte de temps et une communication inefficace entre les différents acteurs.

C'est dans ce contexte que s'inscrit notre projet de fin de module : répondre à un besoin critique de visibilité, d'organisation et de gestion des accès. Le projet simule un environnement réel où des administrateurs, des enseignants et des étudiants interagissent avec la même plateforme, chacun disposant de droits et de fonctionnalités adaptés à son rôle, sous la supervision d'une équipe administrative.

1.0.2 Objectif du Projet

L'objectif majeur du projet est de mettre en place une solution logicielle robuste basée sur le framework **Django** (Python), suivant l'architecture **MVT** (Model-View-Template). Plus spécifiquement, il s'agit de :

- Concevoir une base de données relationnelle capable de stocker l'ensemble des entités scolaires (étudiants, parents, enseignants, départements, matières, examens, jours fériés) avec leurs relations (OneToOneField, ForeignKey).
- Développer une interface web responsive et ergonomique en intégrant le template Bootstrap *PreSkool*, offrant un tableau de bord, des tableaux interactifs et des graphiques dynamiques.

- Implémenter un système d'authentification personnalisé avec trois rôles distincts (Administrateur, Enseignant, Étudiant) et un mécanisme de contrôle d'accès par décorateur (`@role_required`).
- Fournir des opérations CRUD complètes pour chaque module, permettant à l'administration de gérer efficacement l'ensemble des ressources humaines et pédagogiques de l'établissement.

1.0.3 Problématique

La problématique principale est la suivante : comment concevoir une application Web centralisée permettant de gérer l'ensemble des entités d'un établissement scolaire (étudiants, enseignants, départements, matières, examens, jours fériés) en garantissant la cohérence des données, la prévention des incohérences relationnelles et le respect de droits d'accès différenciés selon les rôles (Étudiant, Enseignant, Administrateur) ?

Pour répondre à cette problématique, le projet met en place une base de données relationnelle structurée avec l'ORM Django, un modèle utilisateur étendu (`CustomUser` héritant de `AbstractUser`), un cycle de vie clair des entités scolaires, un mécanisme de vérification automatique des relations (relation `OneToOneField` entre Student et Parent avec suppression en cascade), ainsi qu'un contrôle d'accès via authentification et décorateur personnalisé, complété par l'interface d'administration Django pour suivre et superviser toutes les données.

Un défi particulier réside dans la gestion des relations entre entités et la cohérence des rôles utilisateurs : plusieurs types d'utilisateurs accèdent simultanément à la même plateforme avec des permissions différentes, tandis que certaines opérations (suppression d'un parent, modification d'un étudiant) ont des impacts en cascade sur les données liées. Il est donc impératif de concevoir une architecture qui assure la cohérence des décisions (validation des formulaires, gestion des relations), limite les incohérences liées aux modifications concurrentes, et conserve une traçabilité fiable .

1.0.4 Les Outils utilisés

Python 3.14 : Langage de programmation principal du projet. Python est reconnu pour sa lisibilité et sa richesse en bibliothèques. Il constitue la base du framework Django utilisé pour développer l'application PreSkool.

Django 6.0.3 : Framework web Python de haut niveau suivant l'architecture MVT (Model-View-Template). Il fournit un ORM puissant, une interface d'administration automatique, un système d'authentification intégré et une protection CSRF native.

SQLite : Système de gestion de base de données relationnelle léger utilisé par défaut dans Django pour le développement. Les données sont stockées dans un fichier local `db.sqlite3`, sans nécessiter de serveur séparé.

Bootstrap (Template PreSkool) : Framework CSS/JavaScript utilisé pour l'interface graphique de l'application. Le template PreSkool fournit un design professionnel et responsive avec menu latéral, tableaux interactifs et graphiques dynamiques.

Visual Studio Code est un éditeur de code extensible (Windows, Linux, macOS). Il facilite le développement grâce aux extensions C/C++, au débogage, à l'exécution de tâches et à une navigation rapide dans les fichiers du projet.

Git est un système de gestion de versions distribué. Il permet de suivre l'historique du code, gérer les branches, revenir à des versions stables et faciliter le travail en équipe.

GitHub est une plateforme basée sur Git pour héberger les dépôts, collaborer (issues, pull requests), partager le code et centraliser la documentation du projet.

Chapitre 2

Conception et Développement

2.1 Conception et Développement de l'Application Web de Gestion Scolaire PreSkool

2.1.1 Objectif et principes de conception

Notre projet vise à mettre en place un **système de gestion, de suivi et d'administration des entités scolaires** d'un établissement fictif (étudiants, parents, enseignants, départements, matières, examens, jours fériés), afin de garantir une gestion efficace, centralisée et sécurisée des ressources humaines et pédagogiques. Contrairement à une gestion manuelle ou non centralisée, l'objectif n'est pas uniquement d'enregistrer des données, mais d'assurer une **gestion fiable des relations entre entités** en intégrant les liens entre les modèles (*OneToOneField* entre Student et Parent, *ForeignKey* entre Subject et Department, entre Teacher et Department, entre Exam et Subject) et le **cycle de vie complet des opérations CRUD** (*création, lecture, modification, suppression*) pour chaque module de l'application.

Une attention particulière est portée à la **prévention des incohérences relationnelles** liées aux suppressions et modifications en cascade : le système applique des règles de suppression automatique (`on_delete=CASCADE`) pour maintenir la cohérence des données, par exemple la suppression d'un Parent entraîne automatiquement la suppression de l'Étudiant associé. Enfin, la conception prend en compte la sécurité et la gouvernance via un **contrôle d'accès par rôles** (Étudiant / Enseignant / Administrateur) implémenté à l'aide d'un décorateur personnalisé `@role_required` et d'un modèle utilisateur étendu `CustomUser`, ce qui renforce la séparation des responsabilités et garantit que chaque utilisateur n'accède qu'aux fonctionnalités correspondant à son profil dans l'établissement scolaire.

2.2 Diagrammes UML

2.2.1 Diagramme de classes

Le cœur métier de la plateforme s'articule autour des modèles principaux suivants : `CustomUser`, `Student`, `Teacher`, `Subject`, `Exam`, et `Enrollment`. Les relations principales du système sont :

- **Teacher** (N) — (1) **Department** ;
- **Teacher** (1) — (N) **Subject** ;
- **Subject** (1) — (N) **Enrollment** *(association avec les étudiants)* ;
- **CustomUser (Student)** (1) — (N) **Enrollment** ;
- **Subject** (1) — (N) **Exam** ;
- **Exam** (1) — (N) **ExamResult** ;
- **CustomUser (Student)** (1) — (N) **ExamResult** ;
- **Subject** (1) — (N) **Timetable** *(planification hebdomadaire)*.

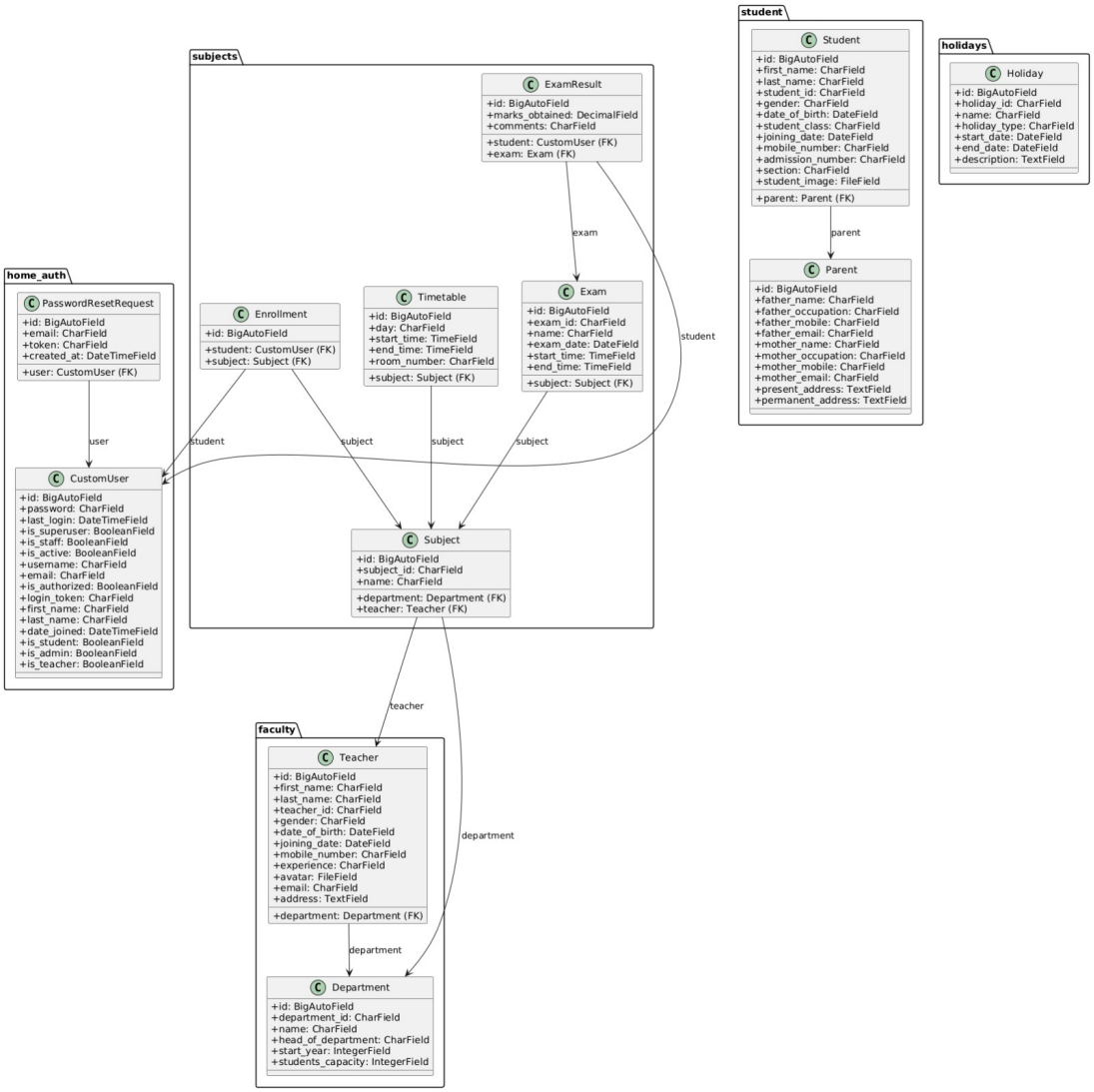

Fig. 2.1: Diagramme de classes du système de gestion scolaire (School Management System), présentant les entités principales (*CustomUser, Teacher, Student, Subject, Exam, etc.*), leurs attributs, et leurs relations (gestion des inscriptions, suivi des examens et emploi du temps).

Structure des fichiers créés

- **home_auth/** (Gestion de l'authentification et des rôles)
  - **models.py** □ Modèle `CustomUser` (rôles admin/teacher/student)
  - **views.py** □ Inscription, Connexion, Déconnexion
  - **decorators.py** □ Stratégie d'autorisation via `@role_required`
- **faculty/** (Gestion du personnel et des départements)
  - **models.py** □ Modèles `Department`, `Teacher`
  - **views.py** □ CRUD Professeurs & Départements, `teacher_dashboard`
- **student/** (Gestion des élèves)
  - **models.py** □ Modèles `Student`, `Parent`
  - **views.py** □ CRUD Élèves, `student_dashboard`
- **subjects/** (Gestion pédagogique)
  - **models.py** □ Modèles `Subject`, `Exam`, `Enrollment`, `ExamResult`, `Timetable`
  - **views.py** □ Inscriptions, notes, vue visuelle de l'emploi du temps
- **holidays/** (Calendrier des événements scolaires)
  - **models.py** □ Modèle `Holiday`
  - **views.py** □ Affichage calendrier dynamique (FullCalendar)
- **templates/** (Vues HTML Django)
  - **Home/base.html** □ Layout principal, Sidebar sécurisée, intégration JS
  - **students/**, **teachers/**, **subjects/** □ Interfaces des sous-systèmes

2.2.2 Structure du projet (résumé)

Le projet suit l'architecture MVT propre à Django (Models–Views–Templates) et est organisé de manière hautement modulaire :

- **Applications Django (Apps) :** `home_auth`, `faculty`, `student`, `subjects`, `holidays`. Chaque application contient :
  - **models.py** : définition des entités métiers (`CustomUser`, `Timetable`, `ExamResult`, etc.).
  - **views.py** : logique métier et traitement des requêtes HTTP (l'équivalent des Controllers).
  - **urls.py** : fichiers de routage internes à chaque application.

- **templates/** : couche présentation (Vues). Contient tous les templates HTML propulsés par le "Django Template Language" (`{% extends %}`, `{{ variable }}`).
- **static/** : ressources front-end.
  - **assets/css/** et **assets/js/** : feuilles de style et logique front-end (Bootstrap, Apex-Charts, scripts de navigation).
  - **assets/plugins/** : librairies externes (ex : Simple-Calendar).
- **school/** : le cœur de configuration global du projet.
  - **settings.py** : configurations de la BDD, des middlewares et de la sécurité.
  - **urls.py** : routeur principal (`web.php` dans Laravel) connectant toutes les apps.

2.2.3 Fichiers importants

- **manage.py** : l'outil CLI de Django (l'équivalent direct de `artisan`). Utilisé pour `migrate`, `runserver`, et la création de super-utilisateurs.
- **db.sqlite3** : la base de données par défaut du projet.
- **requirements.txt** : le registre de dépendances Python (équivalent de `composer.json`).
- **populate_fake_data.py** : script de génération automatisée (Seeders) pour injecter des étudiants, enseignants et notes de test.

Chapitre 3

Cahier des charges

3.1 Contexte et besoin

3.1.1 Objectifs

L'objectif principal de ce projet est de concevoir et développer une application Web permettant la **gestion**, l'**administration** et le **suivi** des entités scolaires d'un établissement fictif nommé PreSkool (étudiants, parents, enseignants, départements, matières, examens, jours fériés). La plateforme doit offrir une gestion centralisée des données scolaires, en garantissant la cohérence des relations entre entités et en assurant un accès différencié selon le profil de chaque utilisateur.

En parallèle, l'application vise à assurer une **gestion transparente** grâce à un suivi clair des opérations CRUD (*création, lecture, modification, suppression*) sur chaque module, et à fournir une **cohérence relationnelle complète** entre les entités. Cette cohérence se traduit par des relations bien définies entre les modèles (relation `OneToOneField` entre Student et Parent, `ForeignKey` entre Subject et Department, entre Teacher et Department, entre Exam et Subject), afin de renforcer l'intégrité des données, la sécurité des accès et la responsabilité dans l'exploitation de la plateforme scolaire.

3.1.2 Acteurs et rôles

L'application prend en charge plusieurs profils d'utilisateurs, chacun disposant de droits spécifiques, afin d'assurer une gestion structurée et sécurisée des données scolaires de l'établissement PreSkool.

Étudiant (Student) L'étudiant accède à l'application après authentification avec son compte personnel. Il dispose d'un tableau de bord dédié (*Student Dashboard*) lui permettant de consulter les informations qui le concernent. Il peut accéder en lecture seule à la liste des étudiants de l'établissement et consulter les détails d'un profil étudiant. L'accès aux fonctionnalités de création, modification ou suppression lui est restreint via le décorateur `role required('student')`.

Enseignant (Teacher) L'enseignant authentifié dispose d'un espace personnel lui permettant de consulter la liste complète des étudiants avec leurs informations détaillées, d'accéder aux matières et aux examens associés à son département, et de consulter le calendrier des jours fériés. Il bénéficie d'un tableau de bord enseignant (*Teacher Dashboard*) centralisant les informations pertinentes à son activité pédagogique. Les opérations de modification et de suppression des données lui sont restreintes, conformément au contrôle d'accès par rôle implémenté via `@role_required(['teacher'])`.

Administrateur (Admin) L'administrateur (rôle `admin`) possède la supervision complète du système. Il gère l'ensemble des entités scolaires : ajout, modification et suppression des étudiants (et de leurs parents en cascade), des enseignants, des départements, des matières, des examens et des jours fériés. Il accède au tableau de bord principal avec les statistiques globales de l'établissement (nombre d'étudiants, départements, enseignants), et peut gérer les comptes utilisateurs via l'interface d'administration Django (`/admin/`). Il supervise également les rôles attribués à chaque utilisateur (`is_admin`, `is_teacher`, `is_student`) et contrôle la cohérence globale des données de l'établissement.

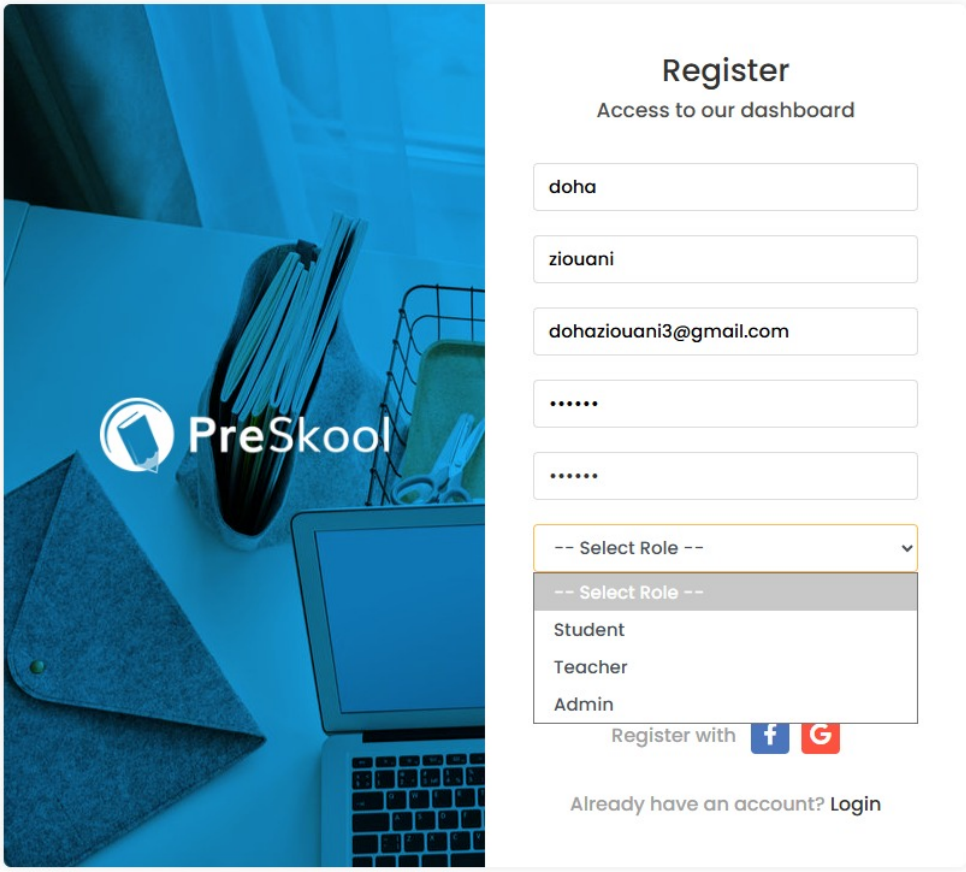

Fig. 3.1: Les rôles de l'application PreSkool

3.1.3 Fonctionnalités principales

Le système propose un ensemble de fonctionnalités couvrant la gestion scolaire, la planification des cours, l'évaluation continue et la supervision globale, afin d'assurer une administration fiable et transparente de l'établissement.

Gestion administrative et pédagogique (Catalogue) L'application permet de gérer l'infrastructure de l'école de manière hiérarchisée : **Départements**, **Professeurs**, **Matières** et **Élèves**. Chaque entité dispose d'une **fiche descriptive** complète (ex : Profil de l'étudiant avec nom, date de naissance, parents, classe, etc.). L'espace d'administration centralisé permet d'ajouter, de modifier et de supprimer ces informations avec fluidité, en liant automatiquement les professeurs à leurs matières respectives et les élèves à leurs classes.

Système d'évaluation et gestion de l'Emploi du temps Les professeurs ont la possibilité de créer et programmer des examens (`Exam`) pour leurs matières. Une fois passés, ils peuvent attribuer des notes et des commentaires détaillés via le module d'évaluation (`ExamResult`). En parallèle, un puissant mécanisme permet de construire de manière dynamique **l'emploi du temps** (`Timetable`) pour chaque matière. Ces plannings récurrents sont affichés aux utilisateurs concernés sous forme d'un calendrier visuel interactif.

Tableaux de bord interactifs (Dashboards) et statistiques Des tableaux de bord dynamiques sont mis à disposition selon les rôles (Admin, Professeur, Étudiant) afin de fournir une vision synthétique de l'état du système : nombre total d'élèves, nombre de départements, métriques de réussites, ou encore un aperçu du planning quotidien. L'interface offre un point de vue immédiat sur "Les meilleurs élèves", "l'Historique des leçons", et l'avancement global de l'établissement en temps réel.

Sécurité et Contrôle d'Accès basé sur les Rôles (RBAC) La sécurité est au cœur du système. Chaque interaction et URL font l'objet d'une **vérification de rôle** stricte (via des décorateurs Django personnalisés comme `@role_required`). Une séparation hermétique garantit que les étudiants ne peuvent que consulter leurs propres données, que les enseignants gèrent leurs classes exclusives, et que seul l'Admin peut réaliser des actes de suppression ou modifier la structure de l'école de manière transverse. Le menu (Sidebar) s'adapte lui-même dynamiquement à ces droits.

3.1.4 Fonctionnalités implémentées

Le système développé permet de couvrir les fonctionnalités essentielles d'un établissement scolaire de bout en bout :

- Authentification centralisée et gestion des utilisateurs (connexion, rôles : Admin, Teacher, Student).

- Ajout, modification, consultation et suppression d'étudiants, professeurs, départements et examens.
- Inscription des étudiants aux matières spécifiques (Enrollment).
- Création d'un emploi du temps interactif et intégration du calendrier des vacances (Holidays).
- Saisie, mise à jour et consultation sécurisées des résultats et des bulletins (Notes).
- Interfaces et tableaux de bord distincts et hautement personnalisés en fonction du profil connecté.
- Sécurisation stricte des accès par rôle (gestion d'Erreur 403 / Access Denied).

Ces fonctionnalités permettent une numérisation centralisée, efficace et sécurisée de l'administration scolaire.

3.1.5 Choix technologiques

Afin de garantir la robustesse, la flexibilité et la maintenabilité de notre application de gestion scolaire (School Management System), plusieurs choix technologiques stratégiques ont été opérés, tant sur le plan de l'architecture backend que sur celui du stockage des données et de l'interface utilisateur.

Le framework backend : Django (Python)

- **Architecture robuste et sécurisée par défaut :** Django est un framework Python de très haut niveau qui encourage un développement rapide et propre grâce à son architecture **MVT** (Modèle-Vue-Template). Il intègre de manière native des protections essentielles contre les failles de sécurité communes (CSRF, XSS, injection SQL).
- **Système d'authentification et rôles :** La gestion des utilisateurs est une composante critique. Django propose un puissant système d'authentification qu'il a été facile d'étendre (via le modèle `CustomUser`) pour gérer distinctement trois profils avec un contrôle d'accès défini (RBAC) : les Administrateurs, les Professeurs, et les Étudiants.
- **L'ORM Django (Object-Relational Mapping) :** Il offre une abstraction permettant de manipuler la base de données sous forme d'objets Python (sans requêtes SQL brutes). Les mécanismes de requêtes et la génération automatique de **Migrations** simplifient drastiquement la maintenance et l'évolution temporelle de la base.

Base de données relationnelle (SGBDR)

- **Cohérence et fiabilité des données :** Le cœur de notre système repose sur des entités fortement interconnectées (Élèves inscrits dans des Cours, appartenant à des Départements, notés par des Professeurs).

Un Système de Gestion de Base de Données Relationnel (via **SQLite** en développement, facilement interfaçable avec **MySQL/PostgreSQL** en production) garantit l'intégrité de ces contraintes et des clés étrangères.

Frontend et utilisation de bibliothèques tierces

Afin d'offrir une interface dynamique et réactive tout en restant sur une architecture principalement "Server-Side Rendered" gérée par les templates Django, l'intégration de bibliothèques frontend s'est avérée cruciale :

- **Bootstrap & jQuery :** Ils permettent de structurer rapidement un design (UI) moderne, réactif (Responsive) et uniforme. `jQuery` assure la manipulation simplifiée du DOM pour fluidifier la navigation du menu interactif (Sidebar), améliorant l'expérience utilisateur.
- **ApexCharts.js :** Ce plugin a été retenu pour l'enrichissement des **Tableaux de bord (Dashboards)**. Il transforme les données statistiques brutes extraites de notre backend en graphiques visuellement impactants.
- **FullCalendar.js :** Indispensable pour notre besoin métier lié à **l'Emploi du Temps (Timetable)**. Il convertit instantanément de simples attributs (heure de début, jour) en un planning d'école visuellement structuré.

Communication Backend-Frontend : Sérialisation JSON

Pour l'affichage dynamique et asynchrone d'événements (comme l'Emploi du temps des professeurs et des étudiants, ou le Calendrier des vacances), nous avons implémenté une communication fluide évitant le temps de latence d'API externes lourdes, grâce à la sérialisation **JSON (JavaScript Object Notation)**.

- L'ORM de Django se charge d'extraire les événements de la table de données pertinente.
- La vue Python rassemble ensuite ces objets et les sérialise en une liste de dictionnaires JSON.
- Ce contexte JSON sécurisé est injecté directement lors du chargement de la page HTML. Les bibliothèques Javascript (comme FullCalendar) consomment alors ce JSON de manière native pour construire le calendrier sans aucun appel réseau asynchrone supplémentaire , ce qui garantit d'excellentes performances d'affichage.

Chapitre 4

Réalisation

4.1 Implémentation

4.1.1 Architecture du projet (MVT)

Contrairement à l'architecture MVC traditionnelle, ce projet suit l'architecture standard de Django appelée **MVT** (Model-View-Template) :

- **models.py** : Définition des entités (CustomUser, Student, Subject, etc.) et de l'ORM.
- **views.py** : Logique de traitement des requêtes (l'équivalent des Controllers).
- **decorators.py** : Filtres de sécurité et permissions (ex : `@role_required`).
- **urls.py** : Définition des points d'entrée (routes) de chaque application.

4.1.2 Fonctionnalités principales

Le workflow principal permet à l'administration de piloter la création des profils (enseignants et élèves), aux professeurs d'attribuer des notes aux examens via un portail dédié, et aux étudiants de consulter leur emploi du temps visuel et leur progression scolaire, le tout centralisé sur des tableaux de bord dynamiques.

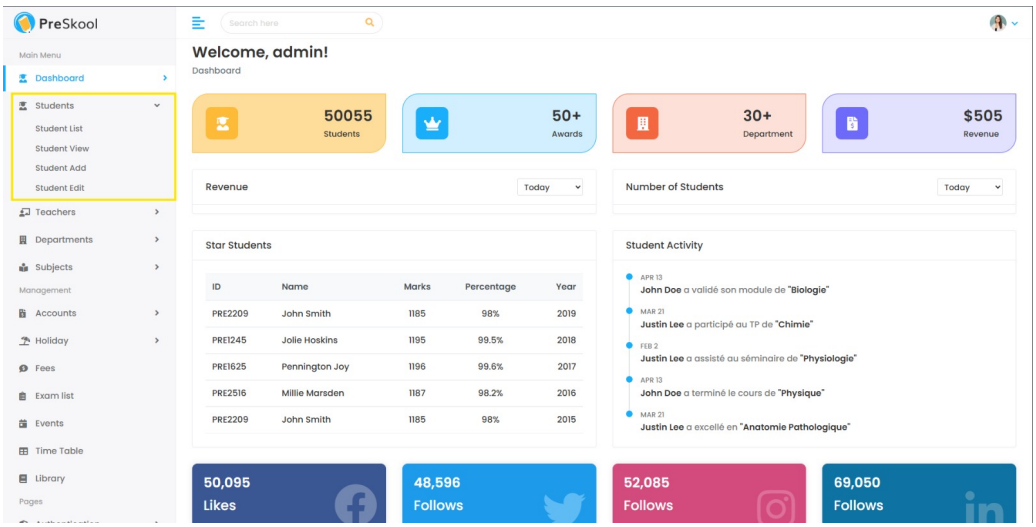

Fig. 4.1: Aperçu du tableau de bord dynamique présentant les statistiques globales

4.1.3 Règles de gestion importantes

Gestion des inscriptions et évaluations

La règle principale de cohérence académique repose sur le couplage strict entre les élèves (`Student`), les matières (`Subject`) et les examens (`Exam`). Une note (`ExamResult`) ne peut être attribuée à un étudiant que si l'examen existe dans une matière précise.

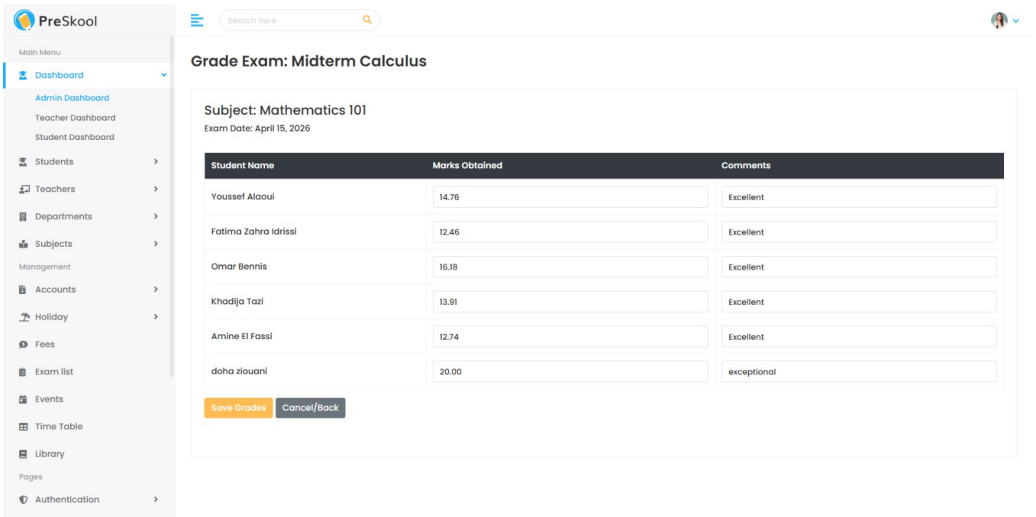

Fig. 4.2: Interface du portail de notation permettant aux enseignants de saisir rapidement les notes et les commentaires de toute une classe pour un examen spécifique

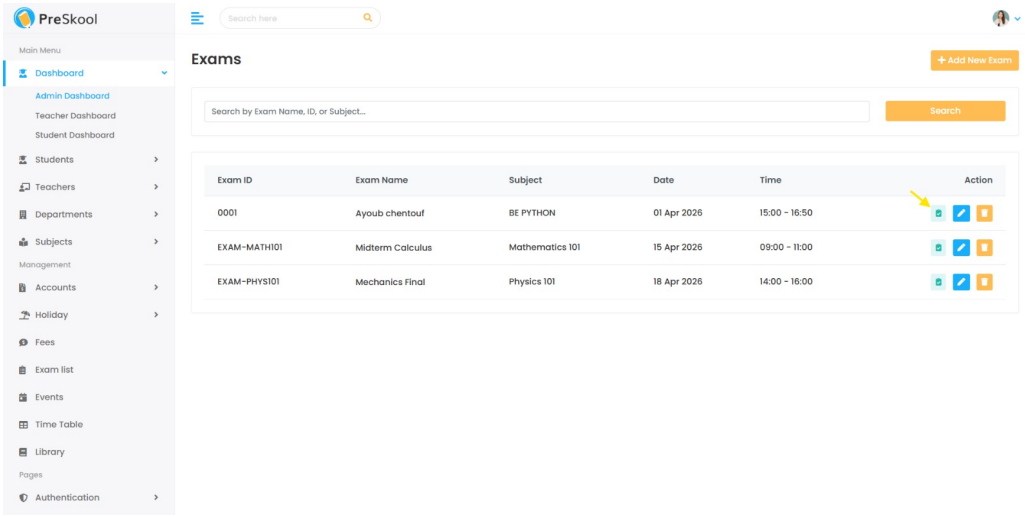

Fig. 4.3: Interface du tableau de bord des examens présentant le nouveau bouton d'accès au portail de notation.

Cohérence de l'emploi du temps (Timetable)

Les sessions de cours sont insérées via le modèle `Timetable` qui définit le jour, l'heure de début et l'heure de fin. Ces données sont ensuite injectées dynamiquement dans *FullCalendar* pour fournir une interface visuelle claire.

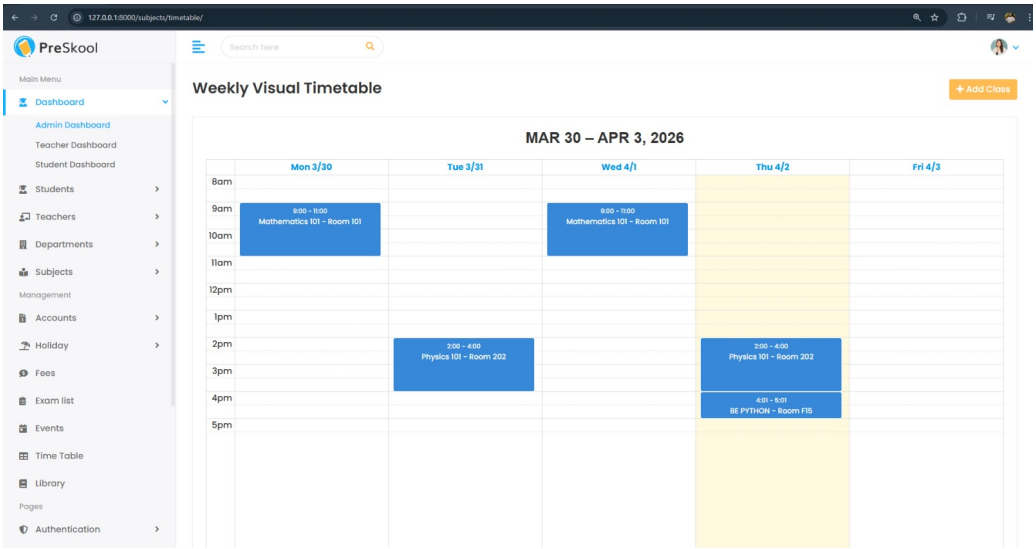

Fig. 4.4: Interface du planning des cours générée via FullCalendar

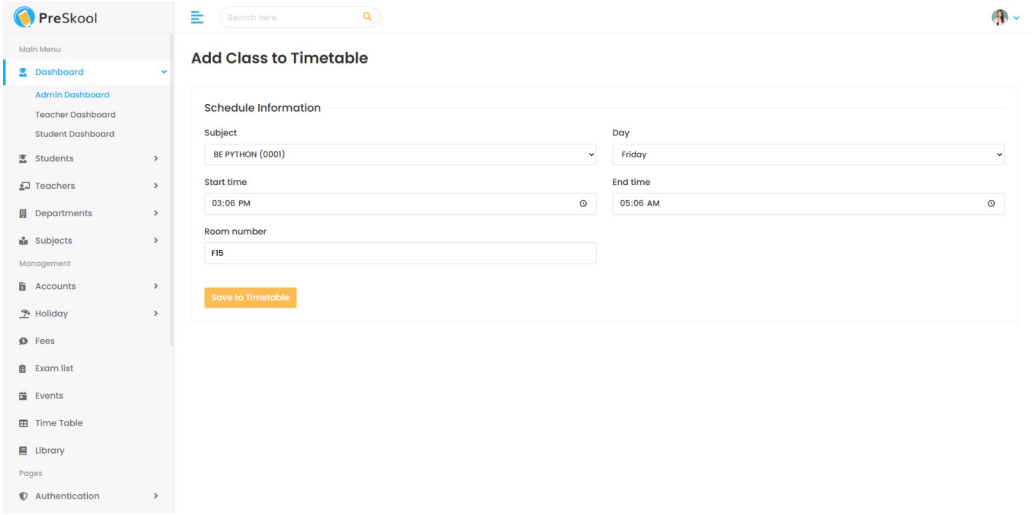

Fig. 4.5: Planifiez de nouveaux cours en sélectionnant la matière, le jour, les heures de début et de fin, ainsi que la salle, puis cliquez sur ’Save to Timetable’ pour mettre à jour l’emploi du temps visuel

Permissions (RBAC - Role-Based Access Control)

Le contrôle d'accès repose sur un modèle hybride strict lié au modèle `CustomUser`. Les actions sont autorisées selon le rôle : l'étudiant consulte ses données, le professeur gère ses classes, et l'administrateur (`Admin`) a un contrôle total.

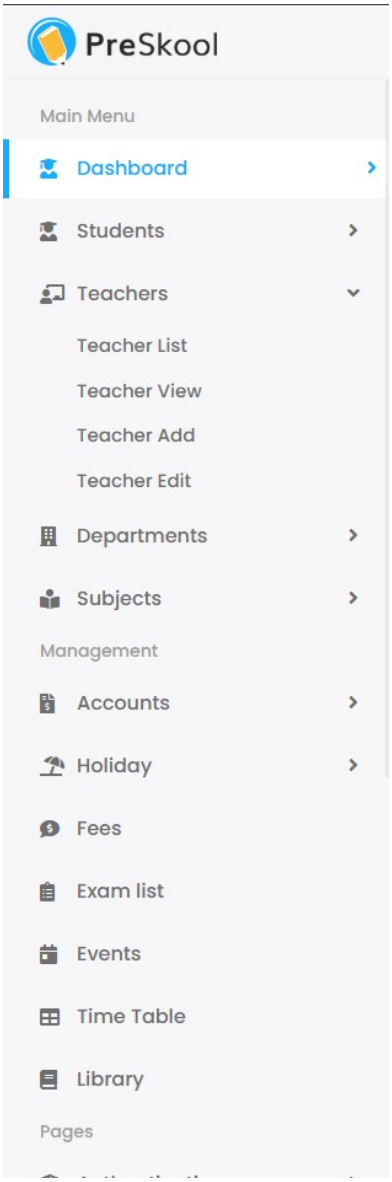

Fig. 4.6: Le contrôle d'accès : view admin

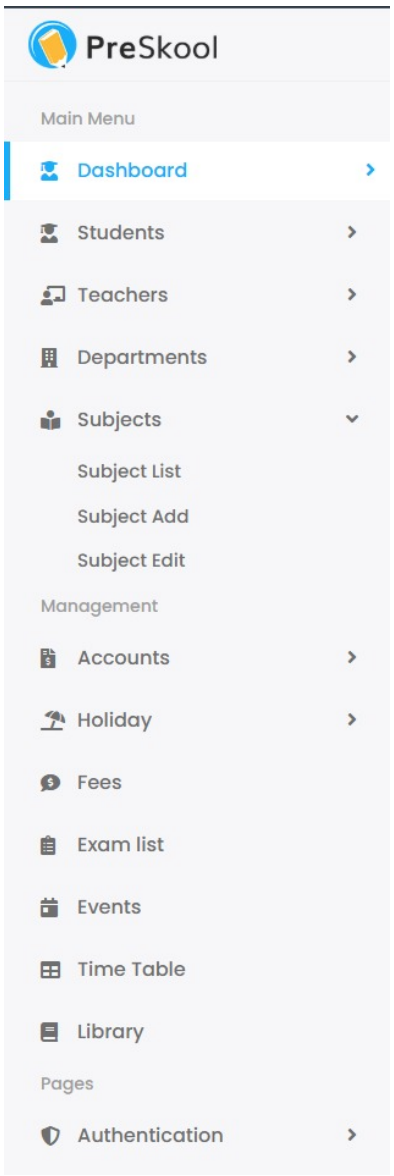

Fig. 4.7: Le contrôle d'accès : view teacher

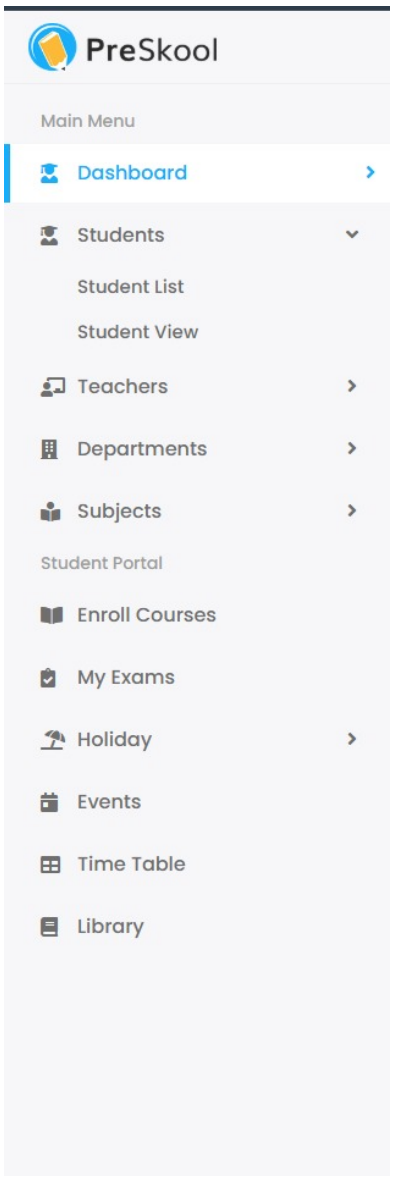

Fig. 4.8: Le contrôle d'accès : view student

4.2 Sécurité et contrôle d'accès

4.2.1 Décorateurs d'authentification

Les fonctionnalités sensibles exigent une session active. Cette protection empêche l'accès direct au portail d'évaluation sans connexion préalable, garantissant ainsi la mécanique native de Django (`@login_required`).

4.2.2 Rôles et permissions (CustomUser)

Les rôles booléens (`is_student`, `is_teacher`, `is_admin`) structurent de manière rigide les droits dans l'application. Le rôle `student` est restreint à la vue (Read-Only) de ses notes et cours. Le rôle `teacher` peut insérer/modifier des évaluations.

4.2.3 Protection des routes d'administration

Les routes de suppression, d'édition et de création sont filtrées par le décorateur `role required('admin')`. Ce *middleware* vérifie la validité du rôle embarqué en session avant d'exécuter la vue.

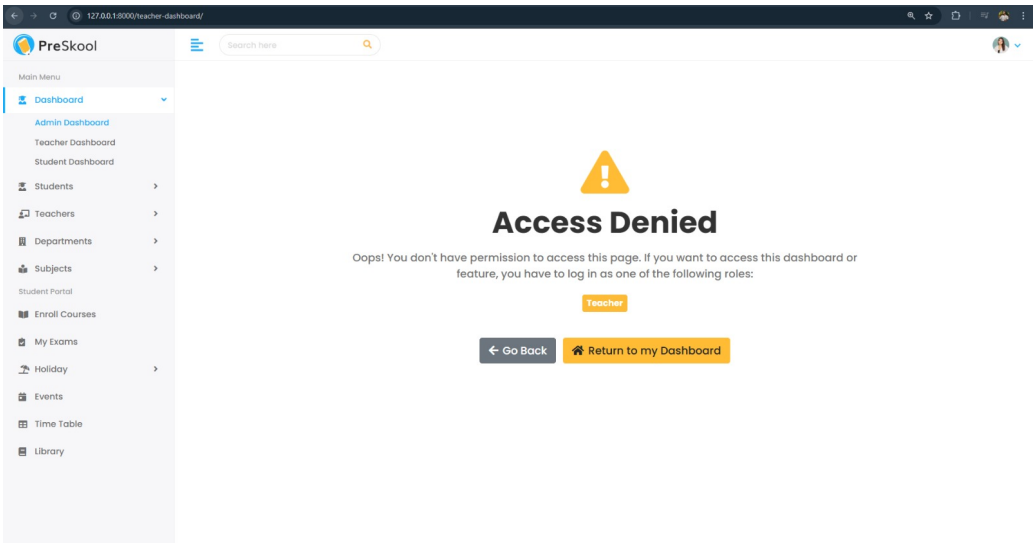

Fig. 4.9: Interception d'une tentative d'accès non autorisée par le système de sécurité

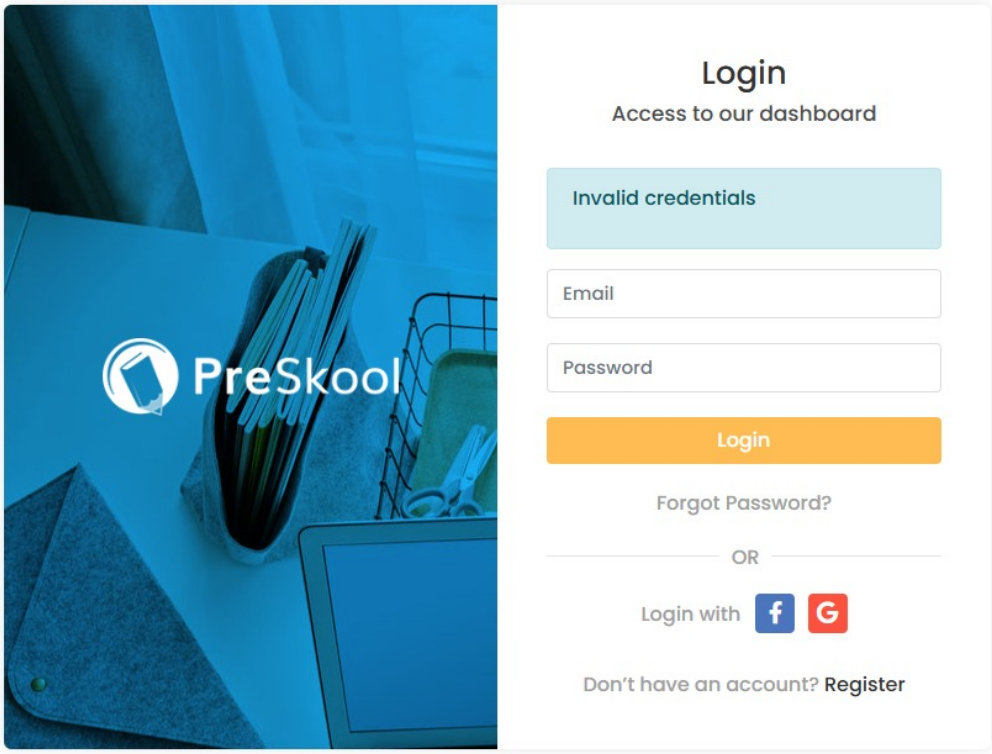

Fig. 4.10: Interface d'authentification illustrant la gestion sécurisée des erreurs avec une alerte 'Invalid credentials' pour bloquer les tentatives de connexion incorrectes.

4.2.4 Validation et traitement des formulaires

Les données envoyées via POST sont interceptées dans les `views.py`. L'ORM Django protège intrinsèquement l'application contre les failles d'injection SQL.

4.2.5 Gestion des sessions

La persistance des sessions est gérée par le framework natif via `django_session`, chiffrant les cookies pour empêcher toute usurpation d'identité.

4.3 Tests et scénarios

4.3.1 Scénarios de test

Les scénarios de test suivants ont été appliqués pour valider le comportement global du système éducatif :

- **Scénario Administrateur (Privilèges élevés)** : L'administrateur remplit le formulaire d'ajout d'un étudiant.
  Résultat attendu : Création des entités `Parent` et `Student`, et affichage dans le système.

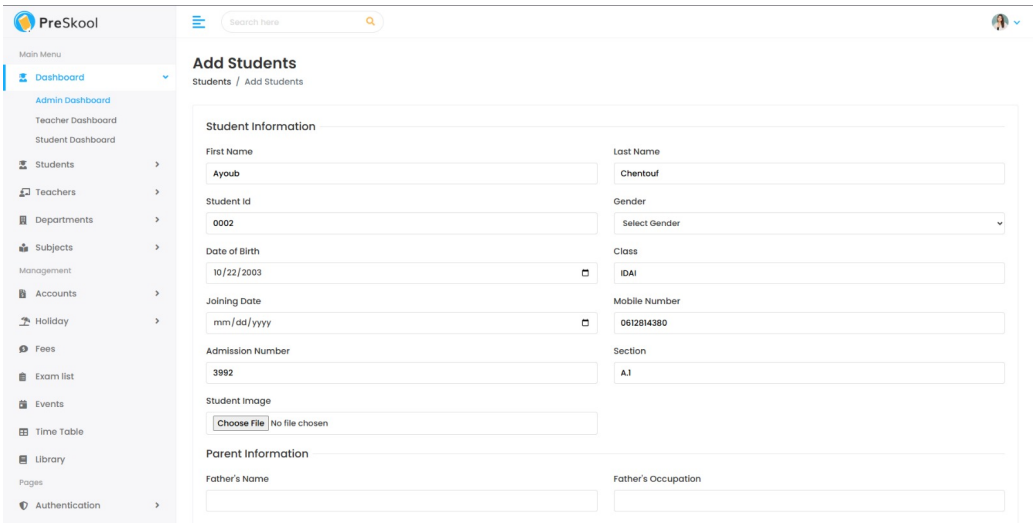

Fig. 4.11: Validation du scénario d'administration : Formulaire d'ajout d'un étudiant

- **Scénario d'évaluation (Professeur)** : Un professeur authentifié accède au portail de la matière "mathematics 101" et saisit une note .
  Résultat attendu : Mise à jour des notes de l'élève en base de données.

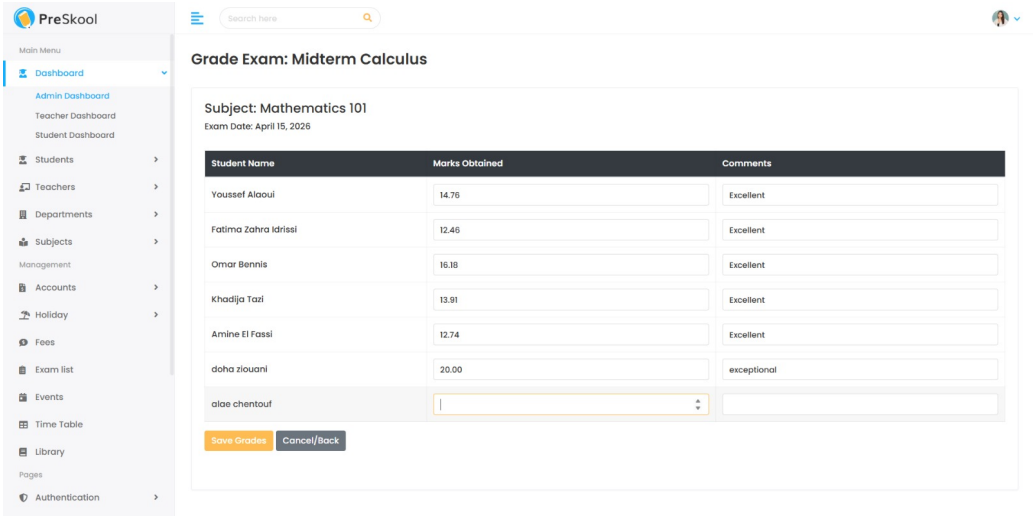

Fig. 4.12: Formulaire d'ajout du note d'un étudiant

Chapitre 5

Difficultés rencontrées

Au cours du cycle de développement de cette plateforme de gestion scolaire, plusieurs défis architecturaux et techniques se sont présentés. Ces difficultés ont nécessité des phases de débogage approfondies et des restructurations de code, renforçant ainsi la robustesse finale de l'application.

5.1 Erreurs

Affichage statique et désorganisé du calendrier dans le tableau de bord étudiant
Lors du développement du tableau de bord étudiant (`/student/dashboard/`), le calendrier s'affichait de manière **statique, mal organisée et hors cadre** : le mois était figé sur **mars 2026**, les jours débordaient de leur conteneur et les événements apparaissaient en dehors du cadre prévu, rendant le calendrier illisible.

La solution a consisté à injecter dynamiquement la date courante depuis la vue Django et à corriger la structure HTML du bloc calendrier pour respecter les contraintes du conteneur Bootstrap.

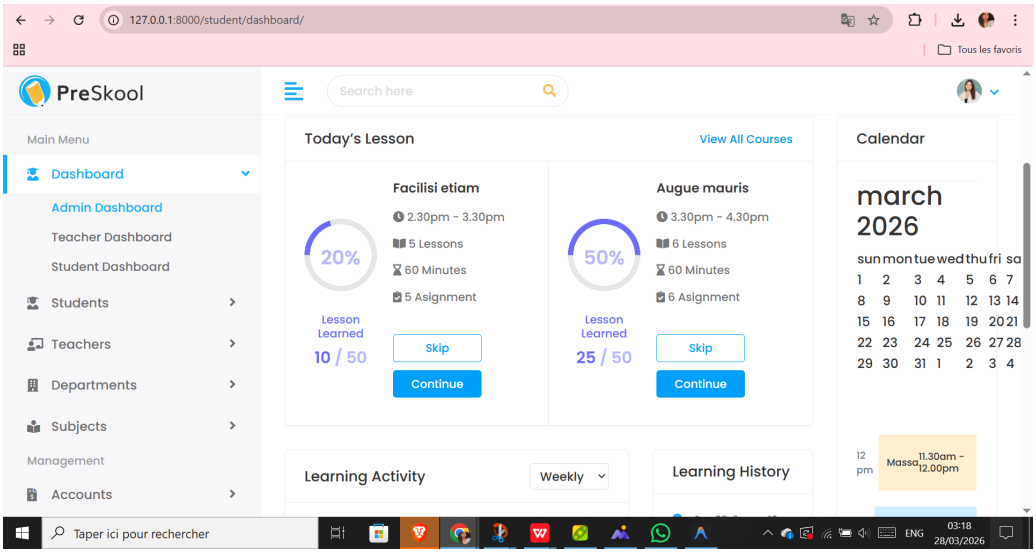

Fig. 5.1: Calendrier statique et mal affiché avant correction

Affichage désorganisé du menu latéral et des fichiers statiques Lors du lancement initial de l'application, la page d'accueil s'affichait de manière **complètement désorganisée et sans style** : le menu latéral apparaissait sous forme de liste non mise en forme, les logos s'affichaient en texte alternatif (*[Logo], [Logo]*), et aucun fichier CSS ou JavaScript n'était chargé, rendant l'interface inutilisable.

La cause était une mauvaise configuration des fichiers statiques dans `settings.py` : les variables `STATICFILES_DIRS` et `STATIC_URL` n'étaient pas correctement définies, et les templates ne contenaient pas la balise `{% load static %}` en haut de chaque fichier HTML.

La solution a consisté à corriger la configuration dans `settings.py` et à ajouter `{% load static %}` en tête de chaque template, en remplaçant tous les liens statiques codés en dur par la syntaxe Django `{% static '...' %}`.

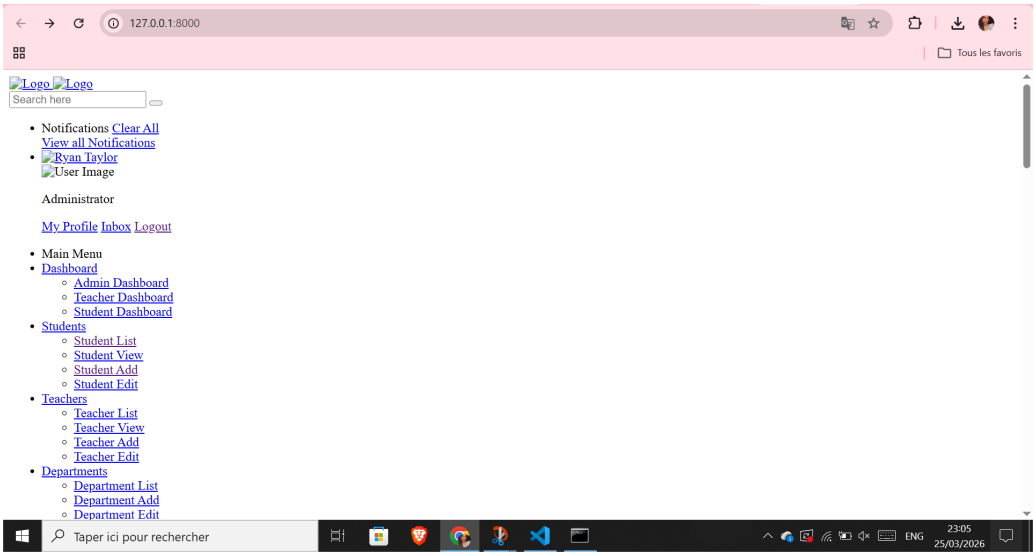

Fig. 5.2: Interface sans style avant correction des fichiers statiques

5.1.1 1. Conflits d'architecture JavaScript et héritage des Templates (Le bug du "#")

Problème :
Lors de l'intégration du thème front-end, un dysfonctionnement majeur de la barre de navigation latérale (*Sidebar*) a été détecté. Le clic sur les menus déroulants n'ouvrait pas les sous-menus, mais ajoutait un symbole "**#**" à l'URL. L'analyse du DOM a révélé que les scripts fondamentaux (`jquery.min.js`, `script.js`) étaient importés de manière redondante à l'intérieur des balises `{% block body %}` de chaque vue enfant, générant des conflits d'écouteurs d'événements et des balises `</body>` dupliquées.

Solution apportée :
Une refonte complète de l'architecture d'héritage des templates Django a été réalisée.

Un script Python d'automatisation a été écrit pour nettoyer toutes les vues enfants, et une structure propre a été imposée : les scripts de base sont désormais chargés exclusivement et une seule fois à la toute fin du fichier `base.html`, via un tag `{% block scripts %}`. Cette approche a définitivement stabilisé l'interactivité de l'interface.

5.1.2 2. Sécurisation stricte des routes et gestion des rôles (RBAC)

Problème :
Le système nécessitant trois types d'utilisateurs mutuellement exclusifs (Admin, Enseignant, Étudiant), la simple vérification `@login_required` de Django était insuffisante. Le risque principal était la vulnérabilité aux attaques par manipulation d'URL (ex : un étudiant tapant manuellement l'URL `/subjects/add/` pour modifier le catalogue de cours).

Solution apportée :
La difficulté a été contournée en développant un décorateur Python sur-mesure (`role required`). Ce *middleware* personnalisé intercepte la requête HTTP, vérifie les attributs booléens du modèle `CustomUser` (`is_student`, `is_teacher`), et redirige impitoyablement toute tentative d'effraction vers une page "Erreur 403 : Accès Interdit", scellant ainsi hermétiquement les processus métiers.

5.1.3 3. Intégration dynamique de l'emploi du temps (FullCalendar.js)

Problème :
La conversion du modèle de base de données relationnelle (`Timetable`) vers un affichage visuel interactif en front-end a posé un défi de sérialisation. La bibliothèque JavaScript *FullCalendar* requiert un format JSON très spécifique pour afficher des événements récurrents hebdomadaires, ce qui diverge de la structure classique d'une réponse ORM.

Solution apportée :
Une logique de transformation de données a été implémentée dans la vue Django (`views.py`). L'ORM extrait les sessions de cours, puis un algorithme Python les formate en une liste de dictionnaires intégrant les jours de la semaine (`daysOfWeek`), délimitant les périodes (`startTime`, `endTime`) et injectant des couleurs dynamiques, format ensuite rendu lisible par le script du calendrier frontend.

5.1.4 4. Cohérence relationnelle dans le module d'évaluation

Problème :
Le développement du portail de saisie des notes (*Grading Portal*) demandait des requêtes croisées complexes. Il fallait s'assurer qu'un professeur ne puisse évaluer *que* les élèves explicitement inscrits (`Enrollment`) dans les matières (`Subject`) qui lui sont personnellement assignées, évitant ainsi des fuites de données inter-classes.

Solution apportée :
L'utilisation avancée de l'API de requêtage SQL de Django (`filter()`, `select_related()`) a permis de croiser avec précision les tables `CustomUser`, `Enrollment` et `ExamResult`. Cela garantit que les formulaires d'évaluation listent exclusivement les bons étudiants pour les bons examens.

Chapitre 6

Conclusion générale

6.1 Conclusion

Ce projet a permis de concevoir et de réaliser avec succès une application Web complète dédiée à la **gestion scolaire (School Management System)**, en couvrant de bout en bout les besoins critiques d'administration des effectifs (élèves, professeurs), de gestion académique (matières, saisie des notes) et de planification (emploi du temps interactif). Grâce à une modélisation relationnelle rigoureuse et à l'utilisation du framework structuré **Django (Python)** couplé à SQLite, la solution mise en place assure un environnement numérique fluide, organisé autour de workflows précis allant de l'inscription documentaire jusqu'à l'évaluation finale de l'étudiant.

En outre, l'application apporte des garanties extrêmement solides en matière de **sécurité** et de **contrôle d'accès**. L'implémentation stricte du contrôle par rôles (*Admin, Teacher, Student*) associée à des décorateurs d'authentification personnalisés (`role required`) protège l'intégrité des données à tous les niveaux. L'intégration de **tableaux de bord (Dashboards) dynamiques** génère en temps réel des indicateurs cruciaux sur les performances éducatives et l'état administratif de l'école. De plus, le calendrier visuel et intuitif (*FullCalendar*) accroît drastiquement la réactivité et la clarté organisationnelle pour tous les membres de la plateforme.

En perspective d'évolution, plusieurs modules additionnels pourraient venir enrichir cet écosystème, notamment l'intégration d'un système de notifications automatisées (alertes par mail en cas de nouvel examen), la génération en un clic des bulletins de notes au format PDF, ou encore le suivi des présences et absences en temps réel à l'aide de lecteurs biométriques ou de codes QR.

Globalement, ce système répond brillamment aux exigences technologiques et fonctionnelles modernes en proposant une gestion centralisée, totalement digitalisée, transparente et hautement sécurisée de l'ensemble de la vie scolaire de l'établissement.